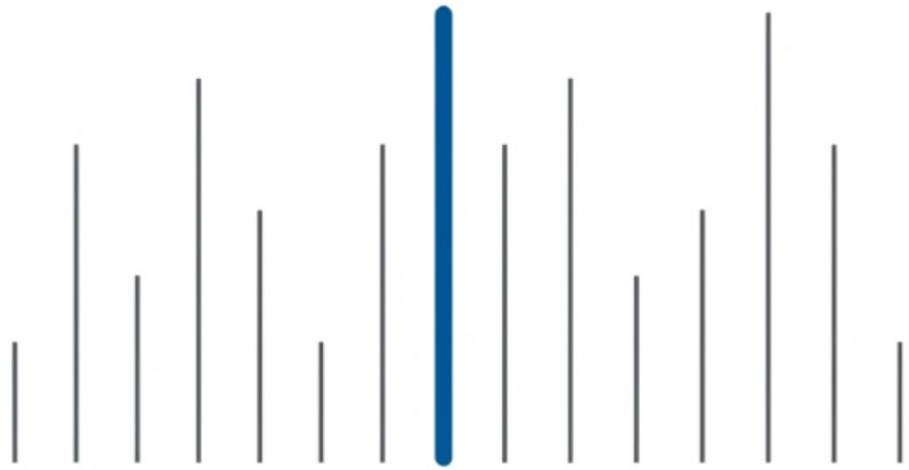
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Algoritmos de ordenamiento:

Heapsort



Objetivos

- Definir rigurosamente la estructura de datos del montículo binario (específicamente el *max-heap*) y demostrar cómo permite un ordenamiento *in-place* garantizando una complejidad temporal en el peor de los casos de $O(n \log n)$.

- Traducir el pseudocódigo algorítmico a C++17 de grado de producción, haciendo uso eficiente de contenedores estándar (std::vector), igestionando adecuadamente la memoria dinámica e interna.

- Contrastar empíricamente Heap Sort vs. Los algoritmos analizados previamente.

Motivación

Hasta este punto en la teoría algorítmica estándar, usualmente nos hemos apoyado en algoritmos de ordenamiento eficientes en la práctica como *Merge Sort* o *Quick Sort*. Sin embargo, ambos presentan compromisos técnicos (*trade-offs*) severos en escenarios críticos:

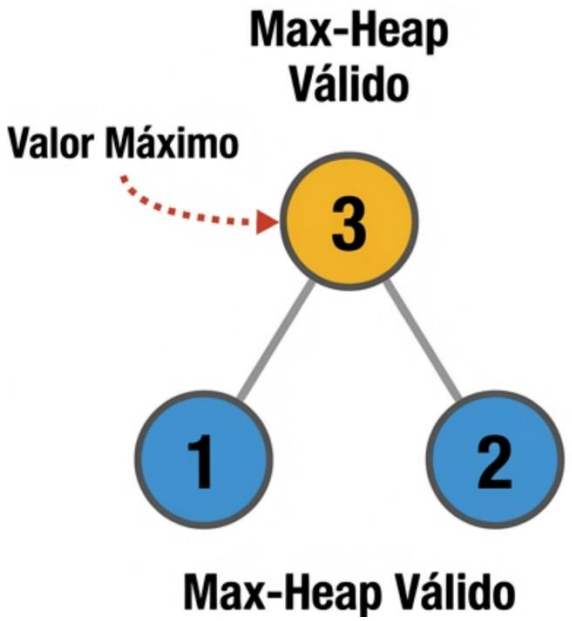
- **Límites Espaciales:** *Merge Sort* requiere memoria dinámica adicional $\Theta(n)$. En sistemas embebidos, de tiempo real o al procesar conjuntos de datos masivos, la latencia de asignación de memoria (*allocation*) y la presión sobre la caché del CPU hacen prohibitivo este costo de espacio.

- **Degradación del Tiempo:** *Quick Sort*, a pesar de su buen rendimiento promedio, está sujeto a un peor caso teórico de $\Theta(n^2)$ si la selección de pivotes es deficiente.

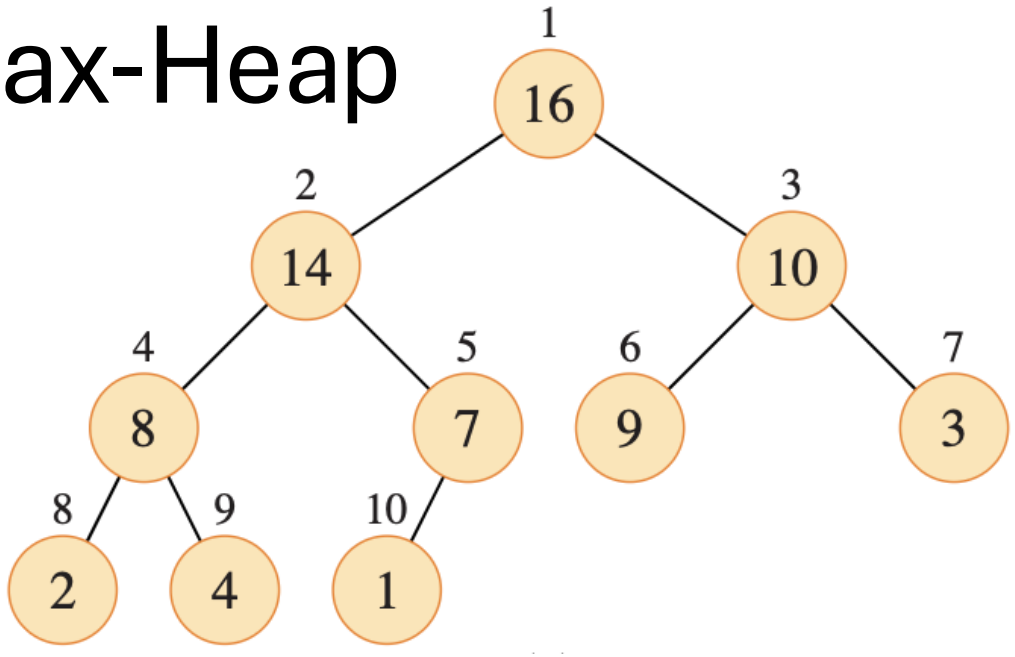
Heap

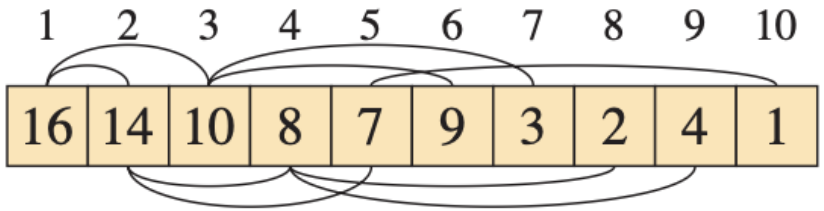
Un *Binary Heap* (montículo binario) es una estructura de datos que se visualiza como un árbol binario casi completo, pero que en la práctica se almacena como un arreglo contiguo A. Para un nodo en el índice i (asumiendo índices basados en 1), sus relaciones son estrictamente algebraicas:

```
PARENT(i)
1    return ⌊i/2⌋

LEFT(i)
1  return 2i

RIGHT(i)
1  return 2i + 1
```



Max-Heap





Los montículos se visualizan conceptualmente como árboles, pero se implementan en la memoria física como arreglos planos.

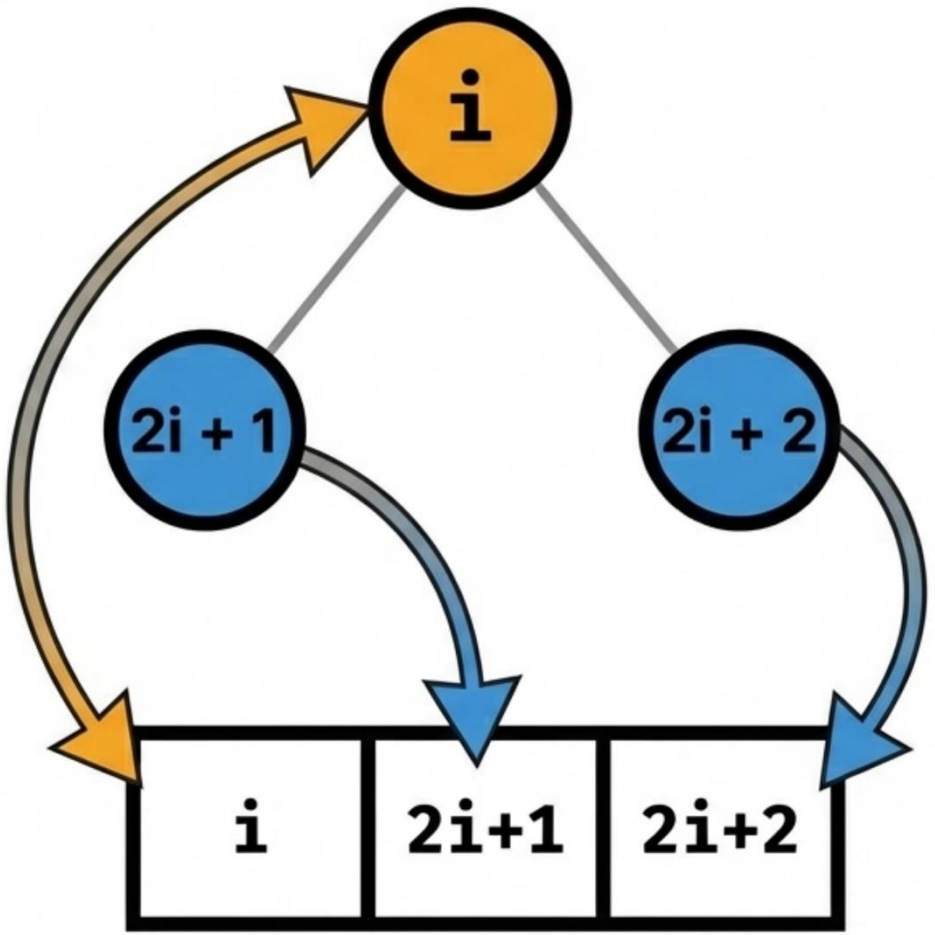
- Cero punteros requeridos.
- Se elimina la sobrecarga de memoria asociada a estructuras de datos dinámicas.
- La relación padre-hijo se calcula mediante álgebra de índices estática.

```
PARENT(i)
1    return ⌊i/2⌋

LEFT(i)
1  return 2i

RIGHT(i)
1  return 2i + 1
```

Heap

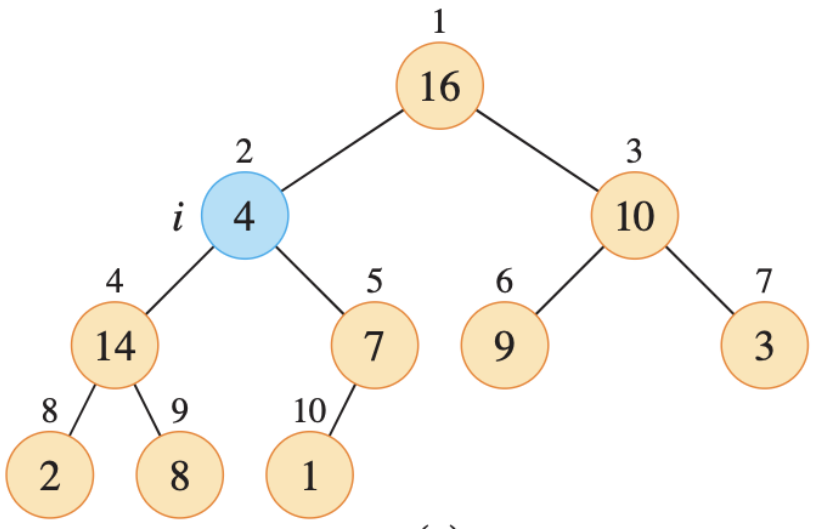
Dado un nodo en el índice i (comenzando desde 0):

```
PARENT(i)
1    return ⌊(i -1)/2⌋

LEFT(i)
1  return 2i + 1

RIGHT(i)
1  return 2i + 2
```

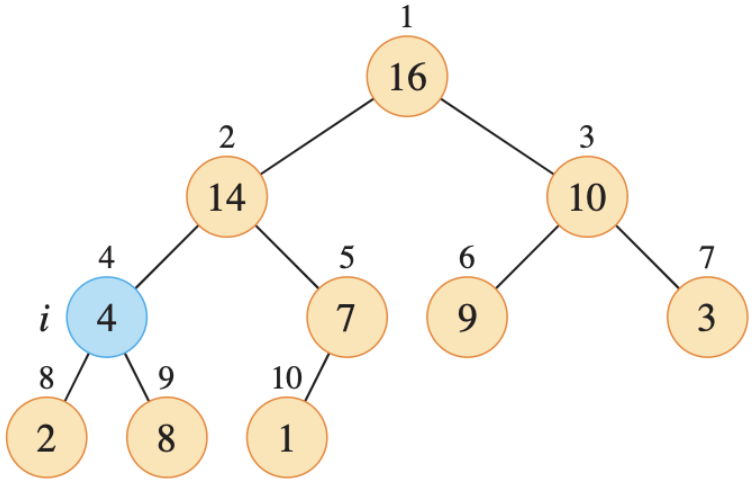


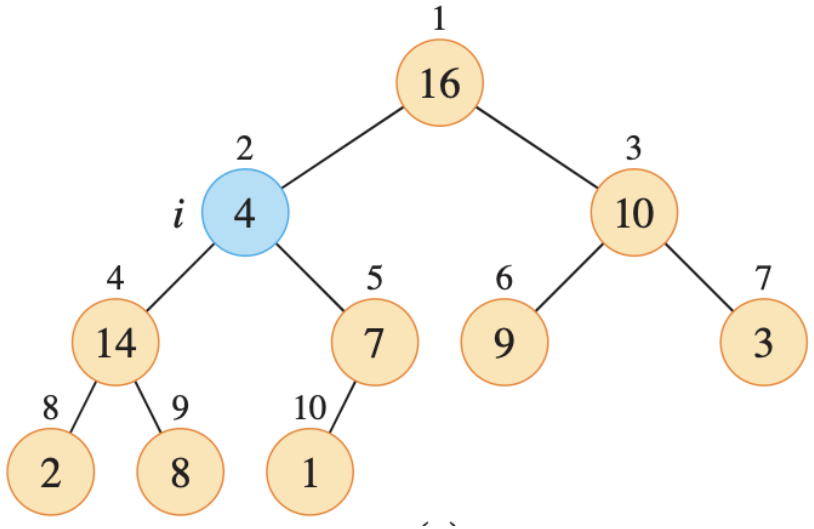
El procedimiento MAX-HEAPIFY mantiene la propiedad max-heap. Al llamar, MAX-HEAPIFY asume que los árboles binarios con raíz en LEFT(i) y RIGHT(i) son max-heaps, pero que $A[i]$ podría ser menor que sus hijos, violando así la propiedad max-heap. MAX-HEAPIFY permite que el valor en $A[i]$ se mueva hacia abajo en el max-heap, de modo que el subárbol con raíz en el índice i cumpla con la propiedad max-heap.

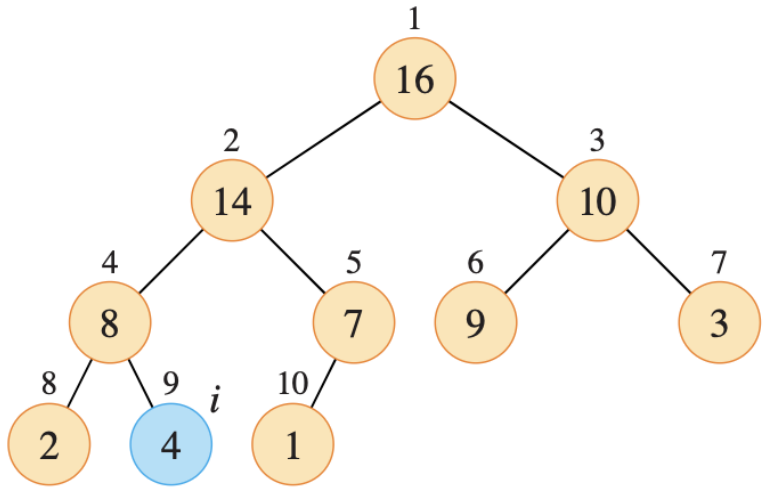
```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```



```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

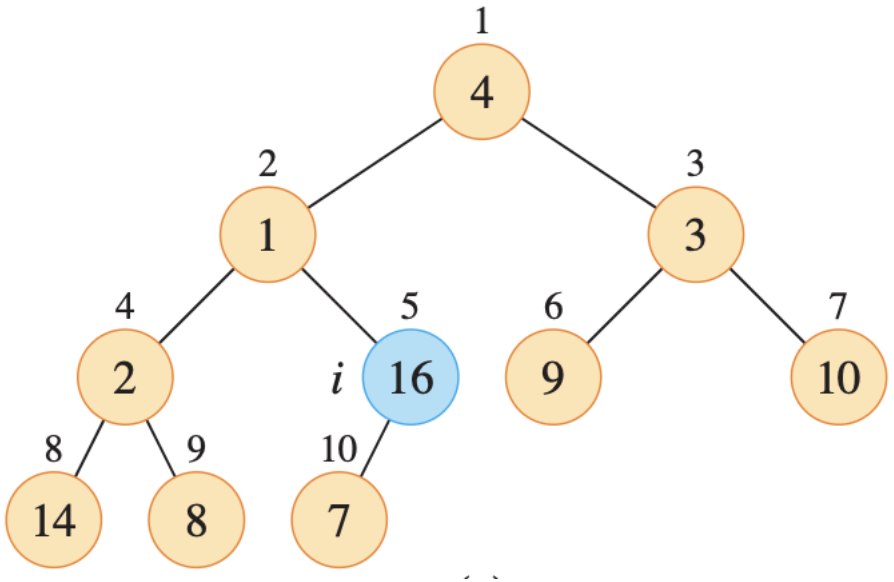






El procedimiento BUILD-MAX-HEAP convierte una matriz $A[1:n]$ en un montón máximo llamando a MAX-HEAPIFY de abajo hacia arriba. Recorre los nodos internos del árbol de abajo hacia arriba (desde $\lfloor n/2 \rfloor hasta 1$). Aunque podría parecer que cuesta $O(n \log n)$, un análisis riguroso (usando series geométricas) demuestra que la mayor parte de los nodos están cerca de las hojas y la altura a recorrer es mínima. Por lo tanto, el costo asintótico real es estrictamente $O(n)$ lineal

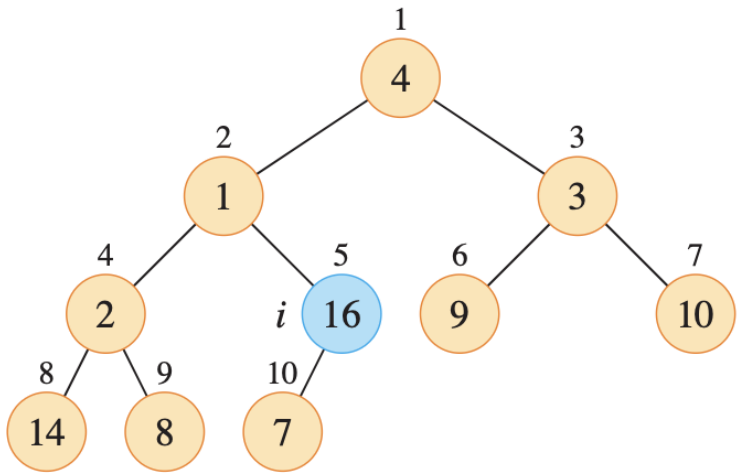
A	4	1	3	2	16	9	10	14	8	7



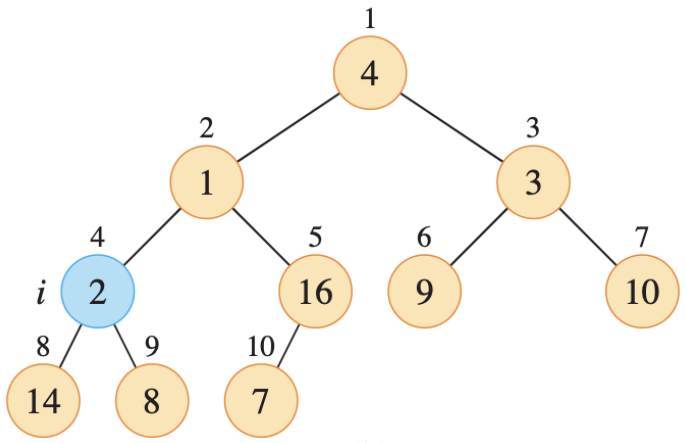

```
BUILD-MAX-HEAP(A, n)
A.heap-size = n
for i = ⌊n/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```

```
BUILD-MAX-HEAP(A, n)
A.heap-size = n
for i = ⌊n/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```

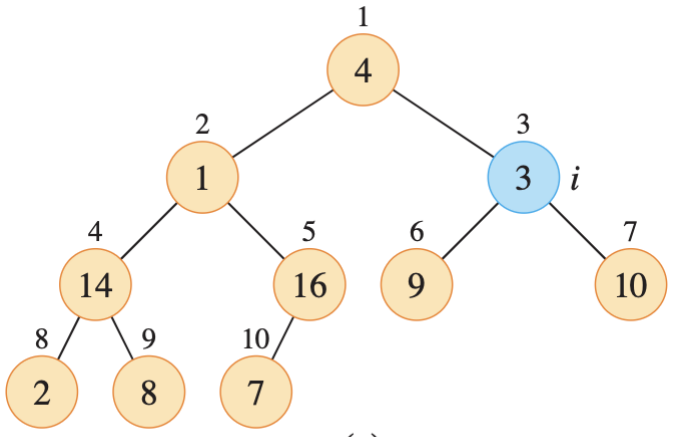
A									
4	1	3	2	16	9	10	14	8	7



(a)

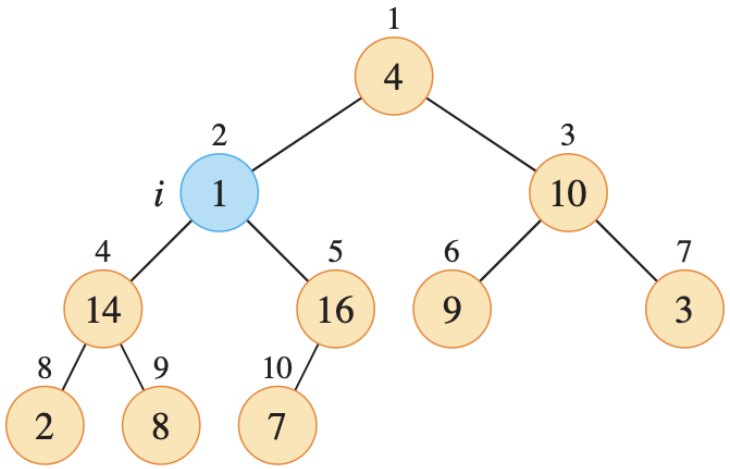


(b)

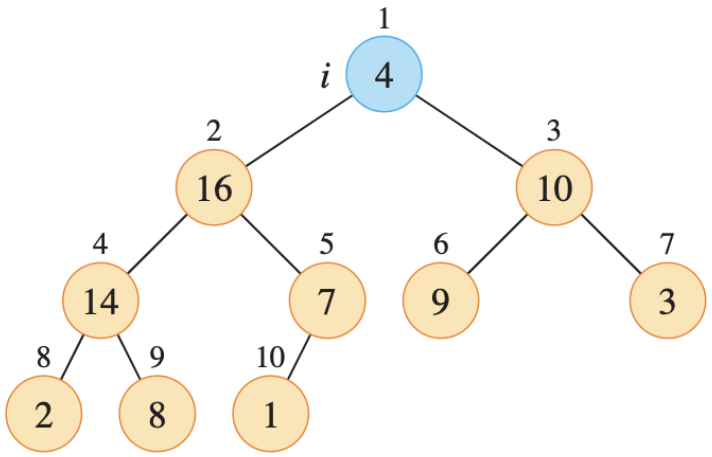


(c)

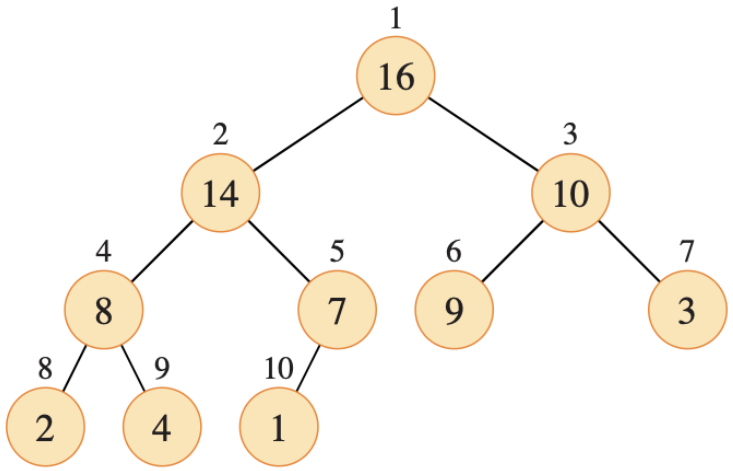
```
BUILD-MAX-HEAP(A, n)
A.heap-size = n
for i = ⌊n/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```



(d)



(e)



(f)

1. Inicialización (El caso base)

Antes de que comience el ciclo, el iterador se inicializa en $i = \lfloor n/2 \rfloor$. En un árbol binario casi completo representado en un arreglo, todos los nodos con índices desde $\lfloor n/2 \rfloor + 1$ hasta n son **hojas**. Un nodo hoja no tiene hijos, por lo tanto, satisface trivialmente la propiedad de *max-heap* (es un montículo válido de tamaño 1).

2. Mantenimiento (El paso inductivo)

El ciclo decrementa i (es decir, avanza hacia la raíz). Cuando estamos evaluando el nodo i, sabemos que los índices de sus hijos son estrictamente mayores que i ($2i$ y $2i + 1$). Como estamos procesando el arreglo en orden inverso (de derecha a izquierda), **los hijos del nodo actual ya han sido procesados en iteraciones anteriores o son hojas**.

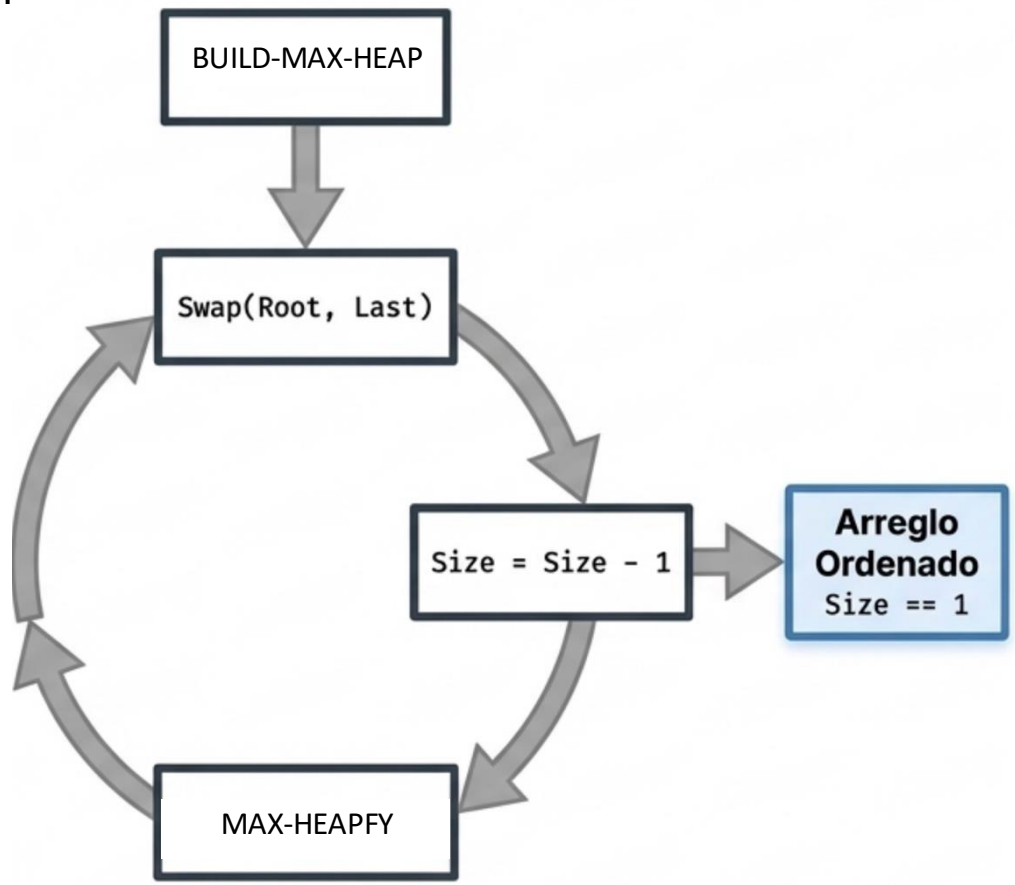
3. Terminación (La conclusión de la prueba)

El ciclo for termina cuando procesa la raíz absoluta del árbol. En base 1 (Cormen), finaliza cuando $i = 0$. En ese punto final, el algoritmo garantiza que cada nodo del árbol, desde el nodo 1 hasta el nodo n, es la raíz de un *max-heap* válido.

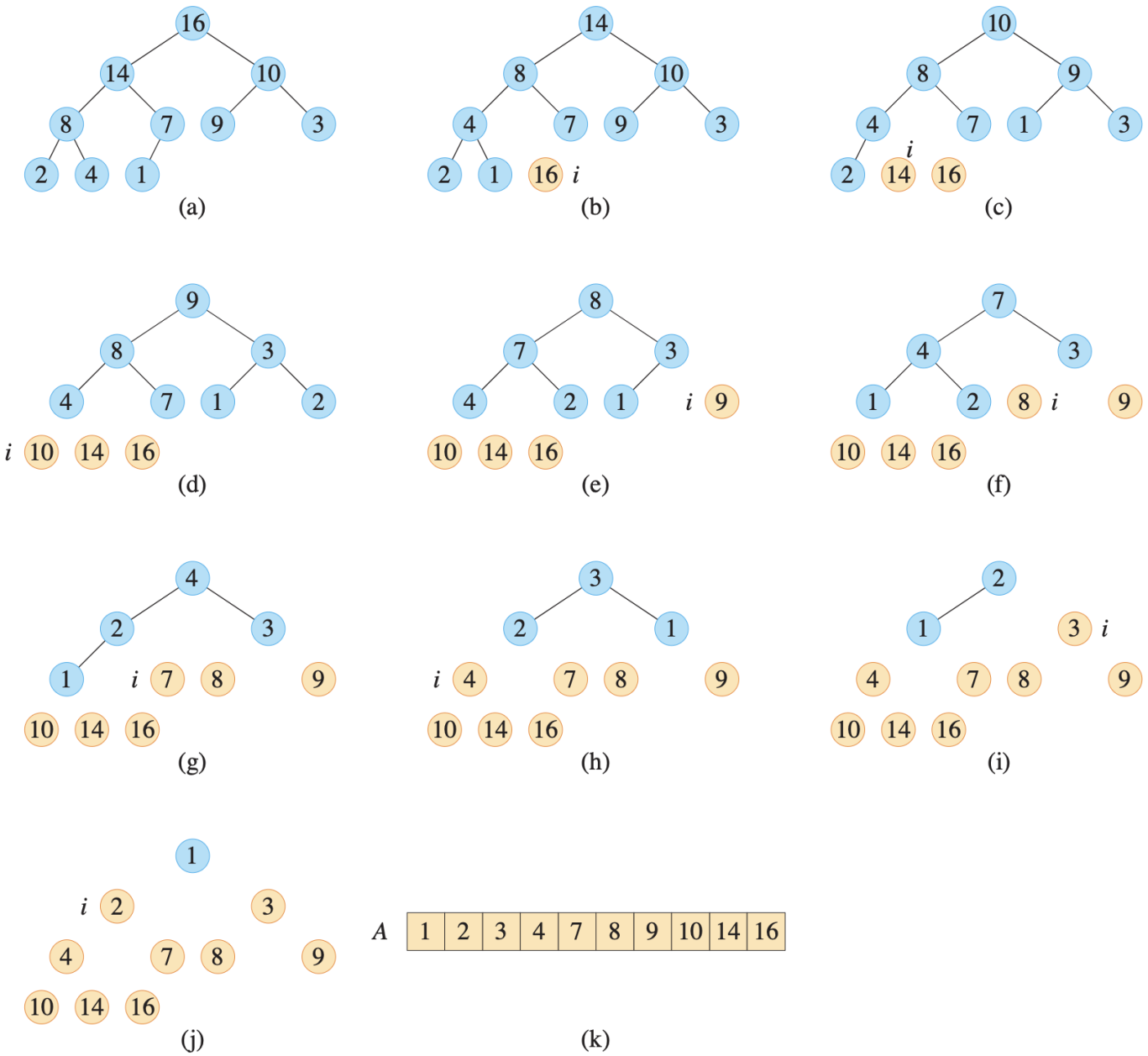
```
BUILD-MAX-HEAP(A, n)
A.heap-size = n
for i = ⌊n/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```

HEAPSORT: El algoritmo primero construye el heap en $O(n)$. Luego extrae el elemento máximo (la raíz A[1]), lo coloca al final del arreglo, reduce el tamaño del heap y llama a MAX-HEAPIFY. Este proceso de extracción ocurre $n-1$ veces, tomando $O(\log n)$ cada uno. Para un complejidad total: $O(n \log n)$ garantizado en el peor caso.

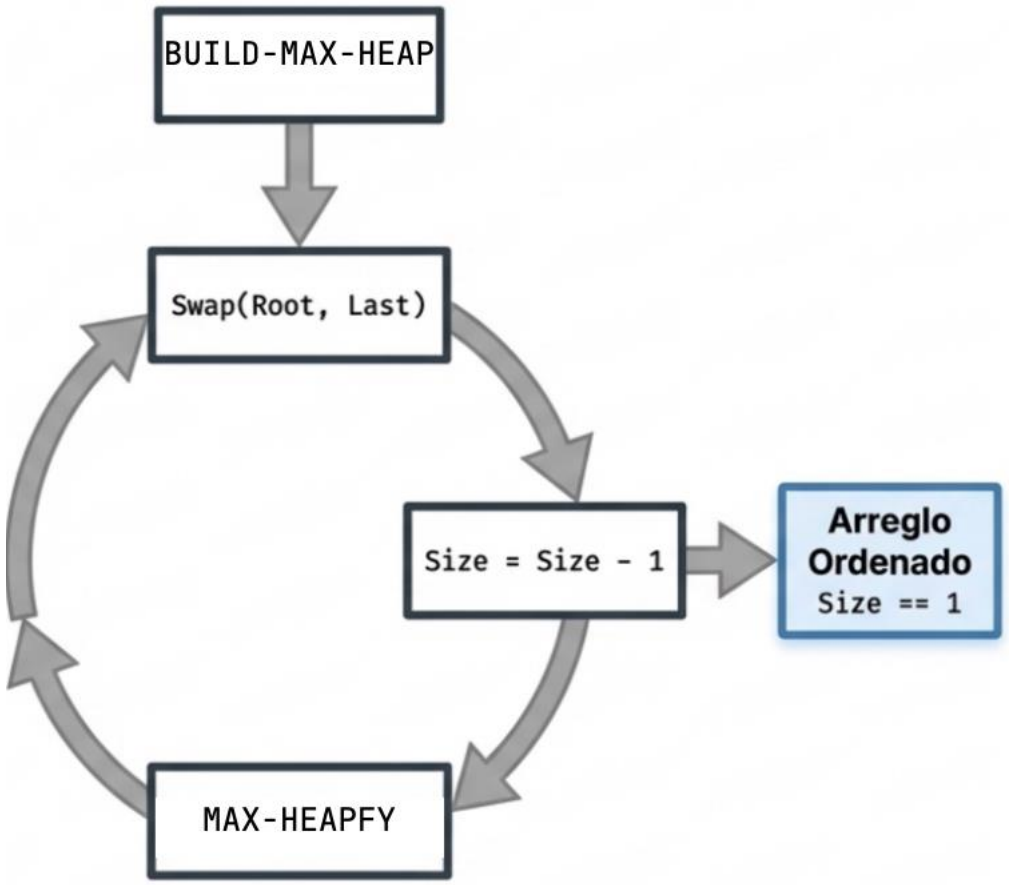
```
HEAPSORT(A, n)
BUILD-MAX-HEAP(A, n)
for i = n downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```

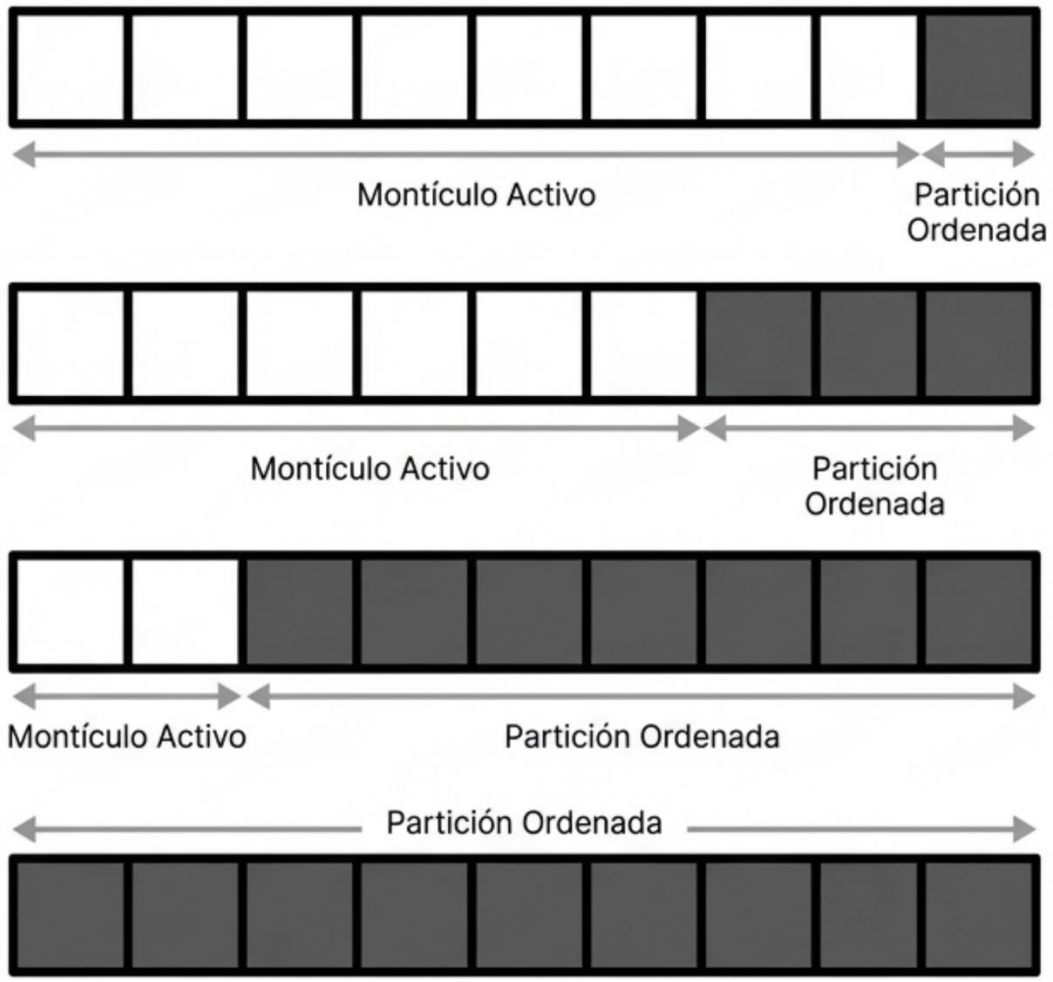


```
HEAPSORT(A, n)
BUILD-MAX-HEAP(A, n)
for i = n downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



Evolución de la memoria durante el ciclo





Algoritmo	Paradigma de Diseño	Tiempo (Peor Caso)	Espacio Auxiliar	Estabilidad	Afinidad de Caché	Resumen Arquitectónico
Heap Sort	Estructura de Datos	Θ(nlogn)	O(1)	Inestable: El proceso de Sift-Down (hundimiento) y el intercambio de la raíz con las hojas destruye irreversiblemente el orden relativo de las claves equivalentes.	Pobre: Los accesos a memoria dictados por la relación 2i+1 y 2i+2 invalidan las líneas de caché pre-cargadas por el procesador.	Transforma un arreglo contiguo en un árbol binario implícito. Maximiza la eficiencia espacial y blinda la cota de tiempo, sacrificando la localidad de referencia. Es el estándar absoluto para sistemas de memoria crítica y como fallback determinista de seguridad.
Insertion Sort	Incremental	$O(n^2)$	O(1)	Estable: Desplaza los elementos estrictamente hacia la derecha sin realizar saltos indexados, preservando la precedencia cronológica original.	Excelente: Iteración y desplazamientos puramente secuenciales, maximizando la utilidad de la caché L1.	Mantiene un sub-arreglo prefijo ordenado, insertando elementos uno a uno. Colapsa computacionalmente a gran escala debido a su naturaleza cuadrática, pero es arquitectónicamente insuperable en datasets casi ordenados o diminutos (n≤16).
Merge Sort	Divide y Vencerás	Θ(nlogn)	Θ(n)	Estable: La rutina de mezcla (merge) prefiere el elemento del sub-arreglo izquierdo ante una igualdad, manteniendo el orden original garantizado.	Buena: Lecturas secuenciales rápidas, pero la constante reubicación de datos entre el arreglo original y el búfer auxiliar castiga el bus de memoria.	Subdivide recursivamente el dominio y amalgama los resultados mediante búferes transaccionales. Su dependencia inherente del asignador de memoria dinámica (malloc / heap allocation) del Sistema Operativo lo descalifica para entornos embebidos o de tiempo real.
Quick Sort	Divide y Vencerás	O(n2)	O(logn)	Inestable: El particionamiento bidireccional en torno al pivote genera intercambios de larga distancia, rompiendo toda estabilidad previa.	Excepcional: El particionamiento in-place escanea la memoria linealmente, saturando el pre-fetcher del CPU a su máxima capacidad.	Emplea punteros convergentes para particionar la memoria. Es el motor estándar de la industria (std::sort) debido a su eficiencia empírica bruta. Su arquitectura exige mecanismos híbridos (IntroSort) que inyecten Heap Sort cuando la profundidad del árbol de recursión excede 2log2n.